



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Symbiosis International (Deemed University)

(Established under section 3 of the UGC Act, 1956)

Re-accredited by NAAC with 'A' grade (3.58/4) | Awarded Category – I by UGC

Founder: Prof. Dr. S. B. Mujumdar, M. Sc., Ph. D. (Awarded Padma Bhushan and Padma Shri by President of India)

Assignment No. 08

Subject:	DevOps Lab
TCODE:	TE7950
Name of Student	Adarsh Jha
PRN No.	23070122261
Branch	CSE, Batch (2023-27), L4
Academic Year & Semester	25 – 26, Sem VII
Date of Performance	23 rd July, 2026
Title of Assignment:	Dockerfile & Image Build
Theory:	Docker is a containerization platform that enables applications and their dependencies to be packaged into lightweight, portable containers. A Dockerfile contains the instructions required to build a Docker image, while a Docker image serves as a template for creating containers. A Docker container is a running instance of an image that provides an isolated environment for executing applications consistently across different systems. In this experiment, a simple Flask web application was containerized by creating the required application files, building a Docker image, running the container, and verifying the application through a web browser. Commands Used • mkdir my-flask-app – Created a new directory for the Flask application. • cd my-flask-app – Navigated to the project directory. • docker build -t my-flask-app . – Built a Docker image named my-flask-app from the Dockerfile. • docker build --no-cache -t my-flask-app . – Rebuilt the Docker image without using cached layers after updating the dependencies. • docker run -p 5000:5000 my-flask-app – Started the Docker container and mapped port 5000 of the container to port 5000 on the host machine for browser access.

Implementation:

Step 1: Create the Flask Application

Created Flask Application — Created the my-flask-app project directory and added the required application files. Implemented the app.py file containing a simple Flask web application that displays the message "Hello, Docker!" when accessed.

Step 2: Create the Requirements File

Created Requirements File — Created the requirements.txt file specifying the Flask dependency required to install the necessary packages for the Flask application.

Step 3 – Create the Dockerfile

Created Dockerfile — Created the Dockerfile specifying the Python base image, working directory, application dependencies, exposed port, and startup command required to containerize the Flask application.

Step 4 – Build the Docker Image

Built Docker Image — Successfully built the Docker image named my-flask-app using the Dockerfile. Docker completed all build stages and created the container image successfully.

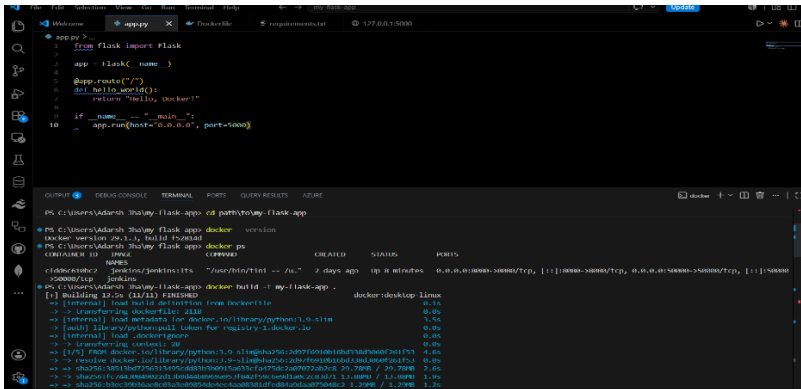
Step 5 – Run the Docker Container

Executed Docker Container — Successfully started the Docker container and launched the Flask application. The application is running on port 5000 and is accessible through the host machine.

Step 6: Verify the Flask Application

Verified Dockerized Flask Application — Opened the Flask application in a web browser using <http://localhost:5000> and successfully verified that the Docker container served the "Hello, Docker!" webpage.

[illegible]

Implementation:	Step 1: Create the Flask Application Created Flask Application — Created the my-flask-app project directory and added the required application files. Implemented the app.py file containing a simple Flask web application that displays the message "Hello, Docker!" when accessed. Step 2: Create the Requirements File Created Requirements File — Created the requirements.txt file specifying the Flask dependency required to install the necessary packages for the Flask application. Step 3 – Create the Dockerfile Created Dockerfile — Created the Dockerfile specifying the Python base image, working directory, application dependencies, exposed port, and startup command required to containerize the Flask application. Step 4 – Build the Docker Image Built Docker Image — Successfully built the Docker image named my-flask-app using the Dockerfile. Docker completed all build stages and created the container image successfully. Step 5 – Run the Docker Container Executed Docker Container — Successfully started the Docker container and launched the Flask application. The application is running on port 5000 and is accessible through the host machine. Step 6: Verify the Flask Application Verified Dockerized Flask Application — Opened the Flask application in a web browser using http://localhost:5000 and successfully verified that the Docker container served the "Hello, Docker!" webpage. 
------------------------	---

The screenshot shows the VS Code interface with a project named 'my-flask-app'. The editor displays the following code in `app.py`:

```
1 from flask import Flask
2
3 app = Flask(__name__)
4
5 @app.route("/")
6 def hello_world():
7     return "Hello, Docker!"
8
9 if __name__ == "__main__":
10     app.run(host="0.0.0.0", port=5000)
```

The terminal output shows the following commands and results:

```
PS C:\Users\Adarsh> flake8 my-flask-app
flake8: no Python files found
PS C:\Users\Adarsh> my-flask-app
Usage: my-flask-app [OPTIONS]
Options:
  -h, --help            Show this message and exit.
  -p, --port PORT        The port to run the application on.
  -H, --host HOST        The host to run the application on.
  -v, --verbose           Enable verbose logging.
  -q, --quiet            Disable verbose logging.
  -d, --debug            Enable debug mode.
  -n, --no-debug         Disable debug mode.
  -s, --ssl-key KEY       The path to the SSL key file.
  -c, --ssl-cert CERT    The path to the SSL certificate file.
  -i, --interface IP      The interface to bind the application to.
  -P, --pid-file FILE     The path to the PID file.
  -f, --force-reload      Force the application to reload.
  -w, --watchdog WHATCHMAN
                        Enable the watchdog.
  -W, --no-watchdog      Disable the watchdog.
  -m, --max-requests MAX_REQUESTS
                        The maximum number of requests to handle before
                        reloading the application.
  -M, --no-max-requests  Disable the maximum requests limit.
  -U, --url-prefix URL_PREFIX
                        The URL prefix for the application.
  -u, --url-base URL_BASE
                        The URL base for the application.
  -e, --environment ENVIRONMENT
                        The environment to run the application in.
  -E, --no-environment   Disable the environment.
  -l, --log-file LOG_FILE
                        The path to the log file.
  -L, --no-log-file      Disable the log file.
  -o, --output-file OUTPUT_FILE
                        The path to the output file.
  -O, --no-output-file   Disable the output file.
  -V, --version           Show the version and exit.
  -h, --help            Show this message and exit.
PS C:\Users\Adarsh> my-flask-app --port 5000
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.17.0.3:5000
Press CTRL-C to quit
172.17.0.1 - - [22/May/2026 07:26:07] "GET / HTTP/1.1" 200
172.17.0.1 - - [22/May/2026 07:26:07] "GET /favicon.ico HTTP/1.1" 404 -
172.17.0.1 - - [22/May/2026 07:26:15] "GET / HTTP/1.1" 200
172.17.0.1 - - [22/May/2026 07:26:15] "GET /favicon.ico HTTP/1.1" 404 -
```

